In this test, the first line defines the stub and the last line sets the expectation. In RSpec, the method being spied on must be declared as a stub before you check to see if it has been received.

Using spies mitigates a common criticism of using test doubles, which is that mock expectations are implicit. It can be difficult to look at a test that sets mock expectations and understand what behavior is being tested. More to the point, it can be difficult to see why a test has failed. The spy explicitly declares the expected behavior. Spies are also more consistent with the given/when/then test structure you've used elsewhere, allowing the stub to be declared in the "given" section and the expectation to be set separately in the "then" part of the test.

Creating Stubs

A stub is a replacement for all or part of an object that prevents a normal method call from happening and instead returns a value that is preset when the stub is created. In RSpec, there are two kinds of stubs. You can create entire objects that exist only to be stubs, which we'll call *full doubles*, or you can stub specific methods of existing objects, which we'll call *partial doubles*.

A partial double is useful, for example, when you want to use a "real" ActiveRecord object but you have one or two dangerous or expensive methods you want to bypass. A full double is useful when you're testing that your code works with a specific API rather than a specific object. By passing in a generic object that responds to only certain methods, you make it hard for the code to assume anything about the structure of the objects being used by the code under test.

Prescription 19	Use partial doubles when you want to ensure most of your real object behavior. Use full doubles when the behavior of the stubbed object doesn't matter—only its public interface does.

Full Doubles

In RSpec, you create full doubles with the `double` method, which is available anywhere. The `double` method takes an optional string argument, which is a name for the double, and then key/value pairs representing messages that can be sent to the double. Since Ruby uses duck typing—which is to say it cares not about the type of objects but only about whether objects respond to the messages sent to them—a stub object created in such a way can be inserted into your application as a replacement for a real object: